

## Week 6: Product Review and Rating Management System

### Scenario:

The e-commerce company wants to collect customer feedback about purchased products. Customers can provide ratings and reviews after completing their orders. The system stores customer opinions and product ratings for analysis.

### Task 1: Create Review and Rating Table

```
SQL> CREATE TABLE Review(Review_ID VARCHAR2(7), Customer_ID NUMBER(10), Product_ID VARCHAR2(7), Rating NUMBER(1), Review_Text VARCHAR2(100), Review_Date DATE, PRIMARY KEY(Review_ID), FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID), FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID));
```

Table created.

```
SQL> DESC Review;
```

| Name        | Null?    | Type          |
|-------------|----------|---------------|
| REVIEW_ID   | NOT NULL | VARCHAR2(7)   |
| CUSTOMER_ID |          | NUMBER(10)    |
| PRODUCT_ID  |          | VARCHAR2(7)   |
| RATING      |          | NUMBER(1)     |
| REVIEW_TEXT |          | VARCHAR2(100) |
| REVIEW_DATE |          | DATE          |

## Store Customer Feedback and Ratings

### Tasks: Add Reviews for Purchased Products.

```
SQL> INSERT INTO Review VALUES ('REV001',1001,'P1001',5,'Excellent laptop performance',DATE '2026-01-20');
```

1 row created.

```
SQL> INSERT INTO Review VALUES ('REV002',1001,'P1002',4,'Good wireless mouse',DATE '2026-01-21');
```

1 row created.

```
SQL> INSERT INTO Review VALUES ('REV003',1002,'P1003',5,'Very good product quality',DATE '2026-02-02');
```

1 row created.

```
SQL> INSERT INTO Review VALUES ('REV004',1002,'P1004',4,'Good quality product',DATE '2026-02-03');
```

1 row created.

```
SQL> INSERT INTO Review VALUES ('REV005',1003,'P1001',5,'Excellent product',DATE '2026-02-15');
```

1 row created.

Select query:

```
SQL> SELECT * FROM Review;
REVIEW_ CUSTOMER_ID PRODUCT      RATING
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
REV001      1001 P1001          5
Excellent laptop performance
20-JAN-26
REV002      1001 P1002          4
Good wireless mouse
21-JAN-26
REVIEW_ CUSTOMER_ID PRODUCT      RATING
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
REV003      1002 P1003          5
Very good product quality
02-FEB-26
REV004      1002 P1004          4
Good quality product
REVIEW_ CUSTOMER_ID PRODUCT      RATING
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
03-FEB-26
REV005      1003 P1001          5
Excellent product
15-FEB-26
```

store customer ratings from 1 to 5 stars.

Ensure this by adding constraint

```
SQL> ALTER TABLE Review ADD CONSTRAINT CHK_Rating CHECK (Rating >= 1 AND Rating <= 5);
Table altered.
```

```
SQL> INSERT INTO Review VALUES ('REV007',1005,'P1003',6,'Very good product',SYSDATE);
INSERT INTO Review VALUES ('REV007',1005,'P1003',6,'Very good product',SYSDATE)
*
ERROR at line 1:
ORA-02290: check constraint (C##NIVI.CHK_RATING) violated
```

## Update Review Comments If Required

```
SQL> UPDATE Review SET Review_Text = 'Good quality but could be improved' WHERE Review_ID = 'REV003';  
1 row updated.
```

## Remove Inappropriate or Invalid Reviews

```
SQL> DELETE FROM Review WHERE Review_ID = 'REV005';  
  
1 row deleted.
```

## Retrieve Product Review Details

write sql queries to:

1.Display all reviews for a product.

```
SQL> SELECT * FROM Review WHERE Product_ID = 'P1001';  
  
REVIEW_ CUSTOMER_ID PRODUCT    RATING  
-----  
REVIEW_TEXT  
-----  
REVIEW_DA  
-----  
REV001      1001 P1001          5  
Excellent laptop performance  
20-JAN-26
```

## 2.Display customer name with their reviews

```
SQL> SELECT C.Customer_Name, R.Product_ID, R.Rating, R.Review_Text, R.Review_Date FROM Customer C JOIN Review R ON C.Customer_ID = R.Customer_ID;
```

| CUSTOMER_NAME | PRODUCT | RATING |
|---------------|---------|--------|
|---------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|                                                   |       |   |
|---------------------------------------------------|-------|---|
| Arun<br>Excellent laptop performance<br>20-JAN-26 | P1001 | 5 |
|---------------------------------------------------|-------|---|

|                                          |       |   |
|------------------------------------------|-------|---|
| Arun<br>Good wireless mouse<br>21-JAN-26 | P1002 | 4 |
|------------------------------------------|-------|---|

| CUSTOMER_NAME | PRODUCT | RATING |
|---------------|---------|--------|
|---------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|                                                          |       |   |
|----------------------------------------------------------|-------|---|
| Priya<br>Good quality but could be improved<br>02-FEB-26 | P1003 | 5 |
|----------------------------------------------------------|-------|---|

|                               |       |   |
|-------------------------------|-------|---|
| Priya<br>Good quality product | P1004 | 4 |
|-------------------------------|-------|---|

| CUSTOMER_NAME | PRODUCT | RATING |
|---------------|---------|--------|
|---------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|           |
|-----------|
| 03-FEB-26 |
|-----------|

|                                         |       |   |
|-----------------------------------------|-------|---|
| Divya<br>Excellent product<br>09-OCT-26 | P1002 | 5 |
|-----------------------------------------|-------|---|

## 3.Find products having maximum reviews

```
SQL> SELECT Product_ID, COUNT(*) AS Total_Reviews FROM Review GROUP BY Product_ID HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM Review GROUP BY Product_ID);
```

| PRODUCT | TOTAL_REVIEWS |
|---------|---------------|
|---------|---------------|

|       |   |
|-------|---|
| P1002 | 2 |
|-------|---|

#### 4. Display recent customer feedback

```
SQL> SELECT * FROM Review ORDER BY Review_Date DESC;
```

| REVIEW_ | CUSTOMER_ID | PRODUCT | RATING |
|---------|-------------|---------|--------|
|---------|-------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|                   |      |       |   |
|-------------------|------|-------|---|
| REV006            | 1004 | P1002 | 5 |
| Excellent product |      |       |   |
| 09-OCT-26         |      |       |   |

|                      |      |       |   |
|----------------------|------|-------|---|
| REV004               | 1002 | P1004 | 4 |
| Good quality product |      |       |   |
| 03-FEB-26            |      |       |   |

| REVIEW_ | CUSTOMER_ID | PRODUCT | RATING |
|---------|-------------|---------|--------|
|---------|-------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|                                    |      |       |   |
|------------------------------------|------|-------|---|
| REV003                             | 1002 | P1003 | 5 |
| Good quality but could be improved |      |       |   |
| 02-FEB-26                          |      |       |   |

|                     |      |       |   |
|---------------------|------|-------|---|
| REV002              | 1001 | P1002 | 4 |
| Good wireless mouse |      |       |   |

| REVIEW_ | CUSTOMER_ID | PRODUCT | RATING |
|---------|-------------|---------|--------|
|---------|-------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|           |
|-----------|
| 21-JAN-26 |
|-----------|

|                              |      |       |   |
|------------------------------|------|-------|---|
| REV001                       | 1001 | P1001 | 5 |
| Excellent laptop performance |      |       |   |
| 20-JAN-26                    |      |       |   |

#### Example query

#### Calculate average product ratings

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY Product_ID;
```

| PRODUCT | AVERAGE_RATING |
|---------|----------------|
|---------|----------------|

|       |     |
|-------|-----|
| P1001 | 5   |
| P1002 | 4.5 |
| P1003 | 5   |
| P1004 | 4   |

Count the number of reviews per product.

```
SQL> SELECT Product_ID, COUNT(*) AS Total_Reviews FROM Review GROUP BY Product_ID;
```

| PRODUCT | TOTAL_REVIEWS |
|---------|---------------|
| P1001   | 1             |
| P1002   | 2             |
| P1003   | 1             |
| P1004   | 1             |

5.retrieve reviews with ratings above 4

```
SQL> SELECT * FROM Review WHERE Rating > 4;
```

| REVIEW_ | CUSTOMER_ID | PRODUCT | RATING |
|---------|-------------|---------|--------|
|---------|-------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|                              |      |       |   |
|------------------------------|------|-------|---|
| REV001                       | 1001 | P1001 | 5 |
| Excellent laptop performance |      |       |   |
| 20-JAN-26                    |      |       |   |

|                                    |      |       |   |
|------------------------------------|------|-------|---|
| REV003                             | 1002 | P1003 | 5 |
| Good quality but could be improved |      |       |   |
| 02-FEB-26                          |      |       |   |

| REVIEW_ | CUSTOMER_ID | PRODUCT | RATING |
|---------|-------------|---------|--------|
|---------|-------------|---------|--------|

| REVIEW_TEXT |
|-------------|
|-------------|

| REVIEW_DA |
|-----------|
|-----------|

|                   |      |       |   |
|-------------------|------|-------|---|
| REV006            | 1004 | P1002 | 5 |
| Excellent product |      |       |   |
| 09-OCT-26         |      |       |   |

Find highest - rated products.

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY Product_ID ORDER BY Average_Rating DESC;
```

| PRODUCT | AVERAGE_RATING |
|---------|----------------|
|---------|----------------|

|       |     |
|-------|-----|
| P1001 | 5   |
| P1003 | 5   |
| P1002 | 4.5 |
| P1004 | 4   |

## Identify products with average rating above 4

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY Product_ID HAVING AVG(Rating) > 4;
```

| PRODUCT | AVERAGE_RATING |
|---------|----------------|
|---------|----------------|

|       |     |
|-------|-----|
| P1001 | 5   |
| P1002 | 4.5 |
| P1003 | 5   |

## Identify highly rated products

## Generate product rating reports

### report : product rating analysis

- 1.display
- 2.Product name
- 3.number of reviews
- 4.average rating

```
SQL> SELECT P.Product_Name, COUNT(R.Review_ID) AS Number_of_Reviews, AVG(R.Rating) AS Average_Rating FROM Product P JOIN Review R ON P.Product_ID = R.Product_ID GROUP BY P.Product_Name;
```

| PRODUCT_NAME | NUMBER_OF_REVIEWS | AVERAGE_RATING |
|--------------|-------------------|----------------|
| Laptops      | 1                 | 5              |
| Mobile       | 2                 | 4.5            |
| TShirt       | 1                 | 5              |
| Shoes        | 1                 | 4              |

### report2:customer feedback analysis

### Find: most reviewed products

```
SQL> SELECT P.Product_Name, COUNT(R.Review_ID) AS Number_of_Reviews FROM Product P JOIN Review R ON P.Product_ID = R.Product_ID GROUP BY P.Product_Name ORDER BY Number_of_Reviews DESC;
```

| PRODUCT_NAME | NUMBER_OF_REVIEWS |
|--------------|-------------------|
| Mobile       | 2                 |
| Shoes        | 1                 |
| TShirt       | 1                 |
| Laptops      | 1                 |

## Highly related products:

```
SQL> SELECT P.Product_Name, AVG(R.Rating) AS Average_Rating FROM Product P JOIN Review R ON P.Product_ID = R.Product_ID GROUP BY P.Product_Name HAVING AVG(R.Rating) >= 4 ORDER BY Average_Rating DESC;
```

| PRODUCT_NAME | AVERAGE_RATING |
|--------------|----------------|
| Laptops      | 5              |
| TShirt       | 5              |
| Mobile       | 4.5            |
| Shoes        | 4              |

## products requiring improvement

```
SQL> SELECT P.Product_Name, AVG(R.Rating) AS Average_Rating FROM Product P JOIN Review R ON P.Product_ID = R.Product_ID GROUP BY P.Product_Name ORDER BY Average_Rating ASC;
```

| PRODUCT_NAME | AVERAGE_RATING |
|--------------|----------------|
| Shoes        | 4              |
| Mobile       | 4.5            |
| TShirt       | 5              |
| Laptops      | 5              |

## report 3: rating distribution number of 5 star ratings

```
SQL> SELECT COUNT(*) AS Five_Star_Ratings FROM Review WHERE Rating = 5;
```

| FIVE_STAR_RATINGS |
|-------------------|
| 3                 |

## Number of 4 star ratings

```
SQL> SELECT COUNT(*) AS Four_Star_Ratings FROM Review WHERE Rating = 4;
```

| FOUR_STAR_RATINGS |
|-------------------|
| 2                 |

## Number of low rated products

```
SQL> SELECT COUNT(*) AS Low_Rated_Products FROM (SELECT Product_ID FROM Review GROUP BY Product_ID HAVING AVG(Rating) < 3);
```

| LOW_RATED_PRODUCTS |
|--------------------|
| 0                  |

## complete rating

```
SQL> SELECT Rating, COUNT(*) AS Total_Ratings FROM Review GROUP BY Rating ORDER BY Rating DESC;
```

| RATING | TOTAL_RATINGS |
|--------|---------------|
| 5      | 3             |
| 4      | 2             |